



CSC 1310, FALL 2026

SORTING ALGORITHM COMPARISON

OBJECTIVE OF THIS ASSIGNMENT



The primary goal & challenge of this assignment is not just learning to write code but in learning about sort algorithms, comparing algorithm efficiencies, and learning to apply parallel programming using OpenMP.

*Modern software is built to run fast and efficiently on multi-core systems, and that means parallel programming is **no longer optional—it's essential**. Learning it early **gives you a serious edge**, making you more competitive, job-ready, and prepared to solve real-world problems at scale.*

Image created with ChatGPT 4.0 https://chatgpt.com/s/m_6868273bc4b48191978c1a469a4cb100

TWO DUE DATES!

- Part 1: The working program code is due on **10/22/2026**
- Part 2: The testing documentation document & lab report is due on **10/27/2026**

IMPORTANT RULES

YOU MAY NOT USE THE FOLLOWING IN YOUR CODE:

You may not use code or syntax in your programming & lab assignments that have not been taught in the class without referencing your source. **You may not ever use the following in your programs:**

- Defining variables throughout a function instead of at the top of the function.
- Defining entire functions (including mutators and accessors) on a single line
- Placing entire if statement including statement attached to if on a single line
- `while(true)` or `while(1)`
Check out <https://sites.tntech.edu/acrockett/2025/08/15/why-is-whiletrue-or-while1-bad-programming-practice/> to see why this is bad programming practice.
- `goto` statements
Check out <https://sites.tntech.edu/acrockett/2025/08/15/why-is-goto-bad-programming-practice-in-c/> to see why this is bad programming practice.
- recursively calling `main`
- `exit()`
- ternary operators (any syntax requiring a '?')
- range-based for loops
- try-catch
- `getline(cin >> ws, variable);`
- lambda functions
- defining all programmer-defined functions above the main function
- code that has no comment block at the top indicating the filename, author, date, and purpose

If you use any of the things listed above or use code that has not been taught in the class (without referencing your detailed source in a comment) then full points will be deducted for that criteria item in the rubric.

GENERATIVE AI POLICY FOR PROGRAM 2 ASSIGNMENT

- You may use generative AI **to help you with troubleshooting the code and help with answering questions in the documentation. However, if you do, you must share your entire generative AI conversation.** If you have OpenAI's ChatGPT or Claude Sonnet, you can share a public link to your chat by clicking "**Share**" in the top right, creating the link, and then put the link in a comment in your source file(s).
 - **You may NOT copy and paste any part of the programming assignment instructions or sample output,** but you may upload your code to aid in troubleshooting.
 - **You may NOT ask generative AI to create functions or any code for you.** This will be considered academic misconduct for this assignment.
 - **You may not show more than 1 to 2 lines of code to another person** (parent, other students – even if they are not in our class) who is not a TA or instructor. This will be considered academic misconduct for this assignment.
- If you get ideas on how to write a portion of your code from somewhere other than your brain, provided CSC 1310 materials, or CSC 1310 TAs/instructors, you must CITE YOUR SOURCE in a comment in that section of your code. You still must be able to explain what the code is doing.

PART 1: THE PROGRAM

PROGRAM OVERVIEW

You will implement multiple **sorting algorithms** and integrate parallelization using **OpenMP**. Some portions of the code, such as reading input from text files, have already been provided to you. The assignment involves:

- Implementing sorting functions in both sequential and parallel versions.
- Testing your implementations on integer and string data.
- Measuring and analyzing the performance of different algorithms under various conditions.
- Documenting your findings based on experimental results.

HOW TO COMPILE

WINDOWS

If you are using the Windows operating system and the GCC compiler (like MinGW), then it already supports OpenMP by default. You only need to include `-fopenmp` when compiling and include the correct libraries.

COMPILING C++ WITH OPENMP

```
g++ -Wall -fopenmp program2.cpp -o prog2
```

RUN YOUR PROGRAM

```
prog2
```

MAC

Using OpenMP on a Mac requires a bit of setup because Apple's default clang compiler does not support OpenMP by default. You'll need to install a version of the compiler that does support OpenMP, like GCC via Homebrew.

IS HOMEBREW ALREADY INSTALLED?

Open Terminal and run

```
which -s brew
```

If Homebrew is installed, you will see the full path to the brew executable, usually:

```
/opt/homebrew/bin/brew (if on Apple Silicon Macs) or
```

```
/usr/local/bin/brew (if on Intel Macs)
```

If Homebrew is not installed, you will see:

```
brew not found
```

INSTALL HOMEBREW (IF IT ISN'T ALREADY INSTALLED)

In the Terminal, run (all on one line):

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

INSTALL GCC (WHICH SUPPORTS OPENMP)

```
brew install gcc
```

This installs a GCC version like gcc-14 or later, depending on the current release.

CHECK THE GCC VERSION

```
ls /opt/homebrew/bin | grep "g++" (if on Apple Silicon Macs) or  
ls /usr/local/bin | grep "g++" (if on Intel Macs)
```

COMPILING C++ WITH OPENMP

If your Homebrew installed GCC 14, the command will be:

```
g++-14 -fopenmp -Wall program2.cpp -o prog2
```

Replace g++-14 with the version Homebrew installed (gcc-13, gcc-15, etc.)

RUN YOUR PROGRAM

```
./run
```

GIVEN CODE

- **program2.cpp**
- **program2.h**
- **numsSmall.txt** – use this text file along with wordsSmall.txt to test your sorting functions to make sure they actually sort correctly
- **wordsSmall.txt** – use this text file along with numsSmall.txt to test your sorting functions to make sure they actually sort correctly
- **numsLarge.txt**
- **wordsLarge.txt**

SPECIFICATIONS

GIVEN CODE

In **program2.cpp**, you are given all the #includes, part of the **main** function, the **readFromFile()** function, the **printVector()** function, and the **scramble()** function.

MAIN FUNCTION

You are given some of this function and then there are detailed comments describing what you should add in this function.

PROTOTYPE THE FUNCTIONS AS YOU IMPLEMENT THEM

In the **program2.h** file, write function prototypes for the functions listed below. All these functions should be template functions because they need to be able to be used with both integer vectors and string vectors.

- **bubbleSort**: The input will be a vector
- **mergeSort**: The input will be a vector, the low index and the high index.
- **parallelMergeSort**: The input will be a vector, the left index, the right index, and a depth integer variable for parallelization
- **quickSort**: The input will be a vector, the left index, and the right index
- **parallelQuickSort**: The input will be a vector, the left index, and the right index
- **merge**: The input will be the vector, the left index, the right index, and the middle index. This function will be called by both mergeSort and parallelMergeSort.

BUBBLE SORT

For the **bubbleSort()** function, you will implement the Bubble Sort algorithm as shown in class and in sample code.

MERGE SORT

For the **mergeSort()** function, you will implement the Merge Sort algorithm as shown in class and example code, which means you will also need a **merge()** function. The **merge()** function is used by both the **mergeSort()** function and the **parallelMergeSort()** function.

PARALLEL MERGE SORT

When you call the **parallelMergeSort()** function from the main function, you will be sending a fourth argument that represents the depth of recursion, which begins at zero. The depth is required in the parallel version of this algorithm because depth controls how deep the parallel recursion goes to prevent creating too many threads, which would end up slowing down runtime instead of speeding up!

After calculating the middle index of the current range to split the vector into two halves, you will then need an if statement to check if the depth is below 4. This is because if we are not too deep in the recursion tree, we allow parallelism. This keeps thread creation under control.

If the depth is **below 4**, then you will have an OpenMP directive that will tell OpenMP to run independent code **sections** in parallel using multiple threads. Refer to the OpenMP slides on “**Task Parallelism with Parallel Sections**”. Each recursive call to **parallelMergeSort()** should be preceded by an OpenMP directive that will define an OpenMP section. This means that the left half of the vector (from left to mid) will be in a new thread (section) and the right half of the vector (from mid+1 to right) will also be in a new thread (section).

If the depth is at 4 or above, then just do the recursive calls to **parallelMergeSort()** without parallelizing them.

QUICK SORT

For the **quickSort()** function, you will implement the Quicksort algorithm as shown in class and example code, which means you will also need a **partition()** function. The **partition()** function is used by both the **quickSort()** function and the **parallelQuickSort()** function

PARALLEL QUICK SORT

In the **parallelQuickSort()** function, only parallelize the quicksort recursive function calls if the number of items to sort is less than 10000. You can do this by adding a conditional clause to the **#pragma** as you see below.

```
#pragma omp task shared(v) if(high-low > 10000)
```

The **if (high-low > 10000)** part says if the condition evaluates to true (high-low is greater than 10,000), then the task is generated normally and it will be executed asynchronously (in parallel at the same time) by the thread pool. If the condition evaluates to false, the overhead of creating a parallel task is evaluated to be too high for the small amount of work. Therefore, the current thread suspends its current task and executes this code block sequentially right away.

The same partition function can be used for both the serial quick sort function and the parallel quick sort function.

INCREMENTAL PROGRAMMING & TESTING

Only implement ONE FUNCTION AT A TIME and then test that function.

Use the smaller text files and call the **printVector()** function to test to make sure the sort algorithm is working. Then, remove the call to **printVector()** and test the function with the larger files.

PART 2: DOCUMENTATION

In a Word document named **program2_documentation.docx**, you must document going through the following steps to test your code **after you know it is working properly**. Read incremental programming & testing above!

1. First, make sure to **disable anti-virus** on your computer and **stop/close all unnecessary programs** from running (close out Teams, Outlook, Discord, etc.). You may keep Microsoft Word or Google Docs open to document the steps below.
2. **Run Sample Output 1** (shown in Sample Output section in this document).
3. Take a **screenshot** of your sample output 1 results and paste into the document.
4. **Analyze your results** and write answers to the questions below in the document.
 - a. Why do you think you got the results that you got?
 - b. What algorithm was the fastest out of the algorithms you implemented?
 - c. What algorithm was the slowest?
 - d. How does your results compare to the sample output provided?
5. **Run Sample Output 2** (shown in next section), including the input validation.
6. Take a **screenshot** of your sample output 2 results and paste into the document.
7. **Analyze your results.**
 - a. Why do you think you got the results that you got?
 - b. How does it compare to the sample output provided?
8. **Run Sample Output 3** (shown in next section).
9. Take a **screenshot** of your sample output 3 results and paste into the document.
10. **Analyze your results.**
 - a. Why do you think you got the results that you got?
 - b. How does it compare to the sample output provided?
11. **Try several different numbers of threads** and figure out what number of threads gave you the quickest runtime for the parallel algorithms. Provide a **screenshot** of the quickest runtime of parallel bubble sort.

PROGRAM REPORT (TO BE SUBMITTED WITH PART 2)

Fill out this program report and take a screenshot of the confirmation page after submitting and name it **prog2report**.

Program Report Link: https://tntech.co1.qualtrics.com/jfe/form/SV_3vXGh5sYqieNStM

SUBMISSION INSTRUCTIONS

Make sure your source and header file have a comment block at the top where you have put your names, the date, and purpose of program.

Create a folder named **yourTTUusername**. Then, create two subfolders under that one named **Part 1** and **Part 2**. See below to see which files should go in each folder. Zip the parent (your username) folder (.zip) and upload it to Canvas for both parts. Note that it is expected that the Part 2 folder may be empty in the part 1 assignment submission but for the Part 2 assignment submission, both Part 1 and Part 2 folders should have files. Ensure you have all these files in your submission by downloading them from Canvas after you submit them and checking the folder!

- **yourTTUusername** folder (zip file of this parent folder submitted for both parts)
 - **Part 1** folder (due 10/22/26)
 - **program2.cpp**
 - **program2.h**
 - **all the provided text files**
 - **Part 2** folder (due 10/27/26)
 - **program2_documentation.docx**
 - **prog2report**

SAMPLE OUTPUT

The number of processors is **machine dependent** as well as the time it will take to run all these algorithms. Therefore, you will likely get different results than what you see in the sample output unless your computer is very similar to Mrs. Crockett's computer running similar processes in the background. Most laptops have 8 or 10 processors.

SAMPLE OUTPUT 1 – USING 4 THREADS

User input highlighted in **yellow**.

```
How many threads do you want OpenMP to use? (Enter 1-500) 4
```

```
Number of Threads Used: 4
```

```
Number of Processors: 14
```

```
What is the name of the string (word) text file?  
(wordsSmall.txt or wordsLarge.txt)
```

```
FILENAME: wordsLarge.txt
```

```
What is the name of the int (nums) text file?  
(numsSmall.txt or numsLarge.txt)
```

```
FILENAME: numsLarge.txt
```

```
SIZE of words: 31100
```

```
SIZE of nums: 37084
```

```
<parallel/algorithm> library sort(int): 0.0048776 seconds
```

```
<parallel/algorithm> library sort(words): 0.0097096 seconds
```

```
Bubble Sort(int): 26.7068 seconds
```

```
Bubble Sort(string): 53.7683 seconds
```

```
Merge Sort(int): 0.0219923 seconds
```

```
Merge Sort(string): 0.0808205 seconds
```

```
Parallel Merge Sort(int): 0.0114647 seconds
```

```
Parallel Merge Sort(string): 0.044354 seconds
```

```
Quick Sort(int): 0.0064787 seconds
```

```
Quick Sort(string): 0.0252117 seconds
```

```
Parallel Quick Sort(int): 0.0085839 seconds
```

```
Parallel Quick Sort(string): 0.0285224 seconds
```

SAMPLE OUTPUT 2 – SHOWING INPUT VALIDATION AND USING 20 THREADS

```
How many threads do you want OpenMP to use? (Enter 1-500) twenty
```

```
Oops! You didn't enter a number!
```

```
Enter a number 1 - 500: duh
```

```
Oops! You didn't enter a number!
```

```
Enter a number 1 - 500: -489
```

```
Oops! You didn't enter a number between 1-500!
```

```
Enter a number 1 - 500: 20
```

```
Number of Threads Used: 20
```

```
Number of Processors: 20
```

```
What is the name of the string (word) text file?  
(wordsSmall.txt or wordsLarge.txt)
```

FILENAME: wordsLarge.txt

What is the name of the int (nums) text file?
(numsSmall.txt or numsLarge.txt)

FILENAME: numsLarge.txt

SIZE of words: 31100

SIZE of nums: 37084

<parallel/algorithm> library sort(int): 0.0052627 seconds

<parallel/algorithm> library sort(words): 0.006366 seconds

Bubble Sort(int): 26.2932 seconds

Bubble Sort(string): 52.9071 seconds

Merge Sort(int): 0.0285785 seconds

Merge Sort(string): 0.0814016 seconds

Parallel Merge Sort(int): 0.0144286 seconds

Parallel Merge Sort(string): 0.0517132 seconds

Quick Sort(int): 0.0066386 seconds

Quick Sort(string): 0.0236089 seconds

Parallel Quick Sort(int): 0.0081306 seconds

Parallel Quick Sort(string): 0.0279783 seconds

SAMPLE OUTPUT 3 – USING 500 THREADS

How many threads do you want OpenMP to use? (Enter 1-500) 500

Number of Threads Used: 500

Number of Processors: 20

What is the name of the string (word) text file?
(wordsSmall.txt or wordsLarge.txt)

FILENAME: wordsLarge.txt

What is the name of the int (nums) text file?
(numsSmall.txt or numsLarge.txt)

FILENAME: numsLarge.txt

SIZE of words: 31100

SIZE of nums: 37084

<parallel/algorithm> library sort(int): 0.122889 seconds

<parallel/algorithm> library sort(words): 0.238648 seconds

Bubble Sort(int): 26.3957 seconds

Bubble Sort(string): 60.6901 seconds

Merge Sort(int): 0.0278909 seconds

Merge Sort(string): 0.096697 seconds

Parallel Merge Sort(int): 0.0170264 seconds

Parallel Merge Sort(string): 0.0652276 seconds

Quick Sort(int): 0.0064127 seconds

Quick Sort(string): 0.0244111 seconds

Parallel Quick Sort(int): 0.0111536 seconds

Parallel Quick Sort(string): 0.0399832 seconds